Capstone Project & Project Guidelines

Participants are **not expected to build enterprise-grade applications**. The focus should be on demonstrating:

- Problem understanding
- Practical use of AI
- Prompt engineering
- Vibe Coding
- API and tool integration
- User experience thinking
- Testing and iteration
- Ability to convert an idea into a working prototype

Each project should result in a **functional Minimum Viable Product (MVP)** that can be demonstrated to evaluators.

PROJECT 1: AI TUTOR

Project Title

AI Tutor: Build a Personalized Learning Assistant

---

1. Project Objective

The objective of this project is to build an AI-powered tutoring application that helps students learn a particular subject or topic through personalized explanations, quizzes, feedback, and recommendations.

Instead of creating a general-purpose AI like ChatGPT, participants should create a tutor focused on a **specific learning area**.

Examples:

- Class 8 Mathematics Tutor
- Spoken English Tutor
- Financial Literacy Tutor
- Excel Tutor
- Python Basics Tutor

- Digital Marketing Tutor
- Interview Preparation Tutor
- Geography Tutor
- Banking Exam Preparation Tutor
- Corporate Training Assistant

The tutor should ideally understand what the learner wants to study, explain concepts at an appropriate difficulty level, ask questions, evaluate answers, and suggest what the student should learn next.

The main goal is to demonstrate how AI can make learning more **interactive, adaptive, and personalized**.

---

2. Target Audience

Participants should clearly define the intended learner before starting development.

Possible target audiences include:

- School students
- College students
- Working professionals
- Competitive examination aspirants
- Employees undergoing corporate training
- Beginners learning software such as Excel or PowerPoint
- People learning a new language
- Professionals preparing for interviews

The project should explain:

Who is the learner?

For example:

“Students aged 13–15 studying Class 8 Mathematics.”

What problem does the learner face?

For example:

“Students often understand formulas but struggle to identify which formula should be used for a particular problem.”

How will the AI Tutor help?

For example:

“The tutor will explain concepts in simple language, provide worked examples, ask practice questions and recommend additional practice when the student makes mistakes.”

---

3. Required Skills

Participants should be able to use:

- Basic Prompt Engineering
- Generative AI tools
- Vibe Coding
- Basic understanding of web applications
- Basic understanding of APIs
- Structured thinking
- User experience design
- Testing and refinement

Advanced programming knowledge should not be required.

Students should rely on AI coding assistants to generate, explain and modify code whenever needed.

---

4. Tools and Technologies

Participants may use tools such as:

AI Model

Any suitable Large Language Model API, such as:

- OpenAI API
- Claude API
- Gemini API

Alternatively, a no-code AI platform may be used.

Vibe Coding / Development

Possible tools include:

- Replit

- Lovable
- Bolt
- Cursor
- Claude Code
- VS Code with an AI assistant

Frontend

A simple web interface can be created using:

- HTML
- CSS
- JavaScript

or generated through a Vibe Coding platform.

Optional Database

If participants want to save learner progress:

- Supabase
- Firebase
- Airtable
- Google Sheets

Database integration should be considered an **optional enhancement rather than a compulsory requirement**.

---

5. Step-by-Step Instructions

Step 1: Select the Learning Topic

Choose one focused learning area.

Avoid extremely broad ideas such as:

“AI Tutor for Everything.”

Instead choose something manageable.

For example:

AI Tutor for Class 8 Mathematics – Algebra

Possible topics:

- Linear equations
- Variables
- Algebraic expressions
- Word problems

A narrow scope makes the project easier to build and demonstrate.

Step 2: Define the Tutor's Personality and Teaching Style

Create a system prompt defining how the tutor should behave.

For example:

The tutor should:

- Explain concepts simply
- Avoid unnecessary technical language
- Ask the learner questions
- Give hints before giving answers
- Encourage the learner to try again
- Adjust difficulty based on performance
- Use examples
- Give short quizzes
- Provide constructive feedback

Participants should experiment with their system prompt until the AI behaves consistently.

Step 3: Create the Main Tutor Interface

Build a simple application containing:

- Application title
- Chat window
- User text box
- Send button
- AI response area

The first version can work exactly like a basic chatbot.

Example:

Student:

“What is a linear equation?”

AI Tutor:

“A linear equation is an equation where the highest power of the variable is 1.

For example:

2x + 5 = 15

Would you like me to explain how to solve it?”

Step 4: Add Learning Modes

Participants should add at least **three learning modes**.

Recommended modes:

Learn

The AI explains the selected concept.

Practice

The AI generates questions.

Quiz

The AI asks multiple questions and evaluates the learner.

Optional additional modes:

- Revision
- Ask a Doubt
- Challenge Mode
- Exam Preparation

Step 5: Add Personalization

The learner should provide some basic information before beginning.

For example:

- Name
- Current level
- Topic

- Difficulty preference

Possible difficulty options:

- Beginner
- Intermediate
- Advanced

The AI should modify its explanations accordingly.

For example:

Beginner:

Use simple explanations and examples.

Advanced:

Use challenging problems and fewer hints.

Step 6: Implement Question Generation

The tutor should be able to generate questions dynamically.

Example:

“Give me five beginner-level algebra questions.”

The AI can generate:

1. $x + 4 = 10$
2. $2x = 14$
3. $3x + 2 = 11$
4. $5x – 7 = 18$
5. $4x + 3 = 19$

Questions should not need to be manually stored.

Step 7: Evaluate Learner Answers

The learner should be able to submit an answer.

The AI should respond with:

- Correct / Incorrect
- Explanation

- Correct answer if necessary
- Hint
- Encouragement

Example:

Student:

“x = 5”

AI:

“Almost correct. Let’s check it together.

2x + 5 = 17

Subtract 5:

2x = 12

Divide by 2:

x = 6

Try the next question.”

Step 8: Track Strengths and Weaknesses

Create a simple learner profile.

For example:

Strong Areas

- Basic equations
- Addition and subtraction

Needs Practice

- Word problems
- Fractions

This information can initially be stored only during the active session.

Persistent storage is optional.

Step 9: Generate a Learning Recommendation

At the end of the session, the AI should generate something such as:

“Today’s Performance”

Questions attempted: 10

Correct: 7

Needs improvement:

- Word problems
- Fractions

Recommended next lesson:

“Linear equations involving fractions”

This provides the personalized learning element of the project.

Step 10: Test the Tutor

Test with different kinds of users.

Try:

- Correct answers
- Incorrect answers
- Completely unrelated questions
- Very easy questions
- Difficult questions
- Spelling mistakes
- Requests for direct answers

Check whether the tutor behaves appropriately.

---

6. Expected Outcomes

The final project should ideally include:

- Working AI Tutor web application
- Personalized AI responses

- Learning mode
- Practice mode
- Quiz mode
- Answer evaluation
- Feedback
- Difficulty selection
- Basic learner progress summary
- Deployed online version

Participants should also present a short demonstration.

Suggested demonstration:

1. Choose a learner level.
2. Select a topic.
3. Ask the tutor to explain it.
4. Attempt a practice problem.
5. Give an incorrect answer.
6. Show how AI provides guidance.
7. Complete a short quiz.
8. Show personalized feedback.

---

7. Tips and Best Practices

Do not attempt to create an AI tutor for every subject.

Start with one subject and approximately 5–10 topics.

Focus on the learner experience rather than complicated technology.

Ask the AI tutor to guide students instead of immediately giving answers.

Add examples wherever possible.

Use structured prompts.

Test your tutor with both correct and incorrect learner responses.

Keep the user interface simple.

A small tutor that works properly is better than a very ambitious application that does not work reliably.

OPTIONAL ADVANCED FEATURES

Participants who finish early may add:

- Voice interaction
- Student login
- Learning history
- Progress charts
- Gamification
- Badges
- Daily learning streak
- PDF notes upload
- Teacher dashboard
- Automatically generated study plans

These are optional and should not be required for successful completion.

PROJECT 2: BUILD AND HOST A VIDEO GAME

Project Title

Vibe Coding Challenge: Build and Host Your Own Online Video Game

---

1. Project Objective

The objective is to use Vibe Coding and AI coding assistants to design, build, test and deploy a simple but enjoyable browser-based video game.

Participants are not expected to become game developers.

Instead, the project demonstrates how someone without a traditional programming background can transform an idea into a functioning software product using AI.

The game should contain:

- A clear objective
- Player controls
- Scoring
- Challenges
- Multiple levels or increasing difficulty

- Win or loss conditions
- Restart capability
- Attractive visuals
- Online hosting

---

2. Target Audience

Participants should identify who would enjoy playing the game.

Possible audiences:

- Children
- Teenagers
- Casual gamers
- Office employees
- Students
- Puzzle enthusiasts

Example:

“A casual browser game for children aged 10–15 where players collect coins while avoiding obstacles.”

---

3. Required Skills

Participants should understand:

- Basic Vibe Coding
- Prompt Engineering
- Basic logic
- Variables
- Conditions
- User interactions
- Testing
- Debugging using AI
- Basic web deployment

Participants should **not be expected to manually write complex JavaScript code**.

Instead they should learn how to clearly describe features to an AI coding assistant.

---

4. Tools and Technologies

Recommended development tools:

- Replit
- Cursor
- Claude Code
- Bolt
- Lovable
- VS Code with AI

Recommended game technologies:

- HTML
- CSS
- JavaScript

Optional:

- Phaser.js

For beginners, plain HTML/CSS/JavaScript is sufficient.

Hosting options may include:

- Replit
- GitHub Pages
- Netlify
- Vercel

---

5. Suggested Game Ideas

Participants should choose a manageable game.

Examples:

Coin Collector

Move a player around the screen, collect coins and avoid obstacles.

Memory Card Game

Flip cards and find matching pairs.

Quiz Adventure

Correct answers allow the player to move through levels.

Catch the Falling Objects

Catch useful items while avoiding dangerous objects.

Maze Escape

Navigate through a maze before time runs out.

Space Shooter

Control a spaceship and destroy incoming objects.

Endless Runner

Jump over obstacles and accumulate points.

Financial Literacy Game

Make financial decisions and try to grow virtual savings.

Cybersecurity Challenge

Identify suspicious emails, passwords and phishing attempts.

Participants may also combine AI learning with games.

Example:

“Math Adventure Game”

Solve maths questions to defeat enemies or unlock levels.

---

6. Step-by-Step Instructions

Step 1: Write the Game Concept

Before coding, participants should write a one-page concept.

Include:

Game name

Target player

Objective

Player controls

Scoring system

How the player wins

How the player loses

Levels

Example:

Game:

“Coin Escape”

Objective:

Collect 20 coins.

Controls:

Arrow keys.

Challenge:

Avoid moving enemies.

Level progression:

Enemies become faster after every level.

Step 2: Ask AI to Create the First Prototype

Start with a simple prompt.

Example:

“Build a browser-based game using HTML, CSS and JavaScript.

The player controls a blue square using arrow keys.

Yellow coins randomly appear on screen.

Every collected coin gives 10 points.

Create a score counter.

Keep the design simple.”

Do not ask AI to build the entire complicated game immediately.

Step 3: Run and Test the First Version

Check:

Does the game open?

Can the player move?

Can objects be collected?

Does the score update?

Fix basic problems before adding features.

Step 4: Improve the Visual Design

Ask the AI coding assistant to improve:

- Background
- Player design
- Buttons
- Typography
- Animations
- Game title
- Start screen

Participants can use AI-generated images where appropriate.

Step 5: Add Obstacles

Add enemies or obstacles.

Example:

“Add three red enemy objects that move randomly.

If the player touches an enemy, reduce one life.”

Step 6: Add Lives

Introduce a simple life system.

Example:

Player starts with:

3 lives.

If the player touches an obstacle:

Lives = Lives – 1.

When:

Lives = 0

show:

“Game Over.”

Step 7: Create Levels

The game should contain at least **three levels or difficulty stages**.

Example:

Level 1

Slow enemies.

Level 2

More enemies.

Level 3

Faster enemies.

Difficulty can also increase automatically based on score.

Step 8: Add a Timer or Challenge Mechanism

Possible challenges:

- Countdown timer
- Limited lives
- Increasing enemy speed
- More obstacles
- Smaller safe area

Example:

“Collect 10 coins within 30 seconds.”

Step 9: Add Start and Game Over Screens

The project should include:

Start Screen

Game title

Instructions

Start Game button

Game Over Screen

Final score

Highest score

Restart button

Step 10: Add Sound

Optional but recommended.

Examples:

- Coin collection sound
- Collision sound
- Game over sound
- Background music

Provide a mute option.

Step 11: Test the Game

Ask multiple people to play it.

Observe:

Are controls clear?

Is the game too easy?

Is it too difficult?

Are instructions understandable?

Are there bugs?

Does restart work?

Step 12: Deploy the Game

The final game must be accessible through a public URL.

For example:

yourgame.example

or through a hosting platform.

Participants should test the deployed version on more than one browser where possible.

---

7. Expected Outcomes

Participants should submit:

- Working browser game
- Public URL
- Game instructions
- At least three levels or increasing difficulty
- Scoring system
- Win/loss condition
- Restart capability
- Basic visual design
- Project explanation

The final report should include a **live gameplay demonstration**.

---

8. Tips and Best Practices

Start extremely simple.

First make the player move.

Then add one feature at a time.

Do not ask AI:

“Build me an amazing professional video game.”

Instead use incremental prompts.

Example:

“Add a score counter.”

Then:

“Add three enemies.”

Then:

“Increase their speed after the player reaches 100 points.”

Always test after every major change.

If something breaks, tell your AI coding assistant exactly what happened.

For example:

“After adding Level 2, the restart button no longer works. Identify the problem and fix it without changing other game behaviour.”

Keep backup versions while developing.

OPTIONAL ADVANCED FEATURES

Advanced participants may add:

- Mobile controls
- Touchscreen support
- Leaderboard
- Multiplayer
- Character selection
- Boss level
- Power-ups
- Achievements
- AI-controlled enemies
- AI-generated questions
- Saving high scores
- User login

These should remain optional.

PROJECT 3: VOICE AGENT FOR CUSTOMER SUPPORT

Project Title

AI Voice Agent for Customer Support

---

1. Project Objective

The objective is to build an AI-powered voice agent that can conduct a basic customer-support conversation over a phone call or voice interface.

The agent should be capable of:

- Greeting the customer
- Understanding spoken questions
- Answering common queries
- Collecting information
- Performing simple predefined actions
- Escalating complex requests to a human

The project should demonstrate how conversational AI can automate simple customer-support interactions.

Participants are **not expected to create a complete call-centre platform**.

The project should focus on one clearly defined business scenario.

---

2. Target Audience

Participants should choose a specific business.

Examples:

- Dental clinic
- Salon
- Restaurant
- Hotel
- Travel agency

- Coaching institute
- Gym
- Yoga studio
- Car service centre
- Real-estate agency
- E-commerce store

Example:

Dental Clinic Voice Agent

The agent could handle questions such as:

“What are your timings?”

“Do you provide root canal treatment?”

“How much does a consultation cost?”

“Where is the clinic?”

“I want to book an appointment.”

---

3. Required Skills

Participants should understand:

- Prompt Engineering
- Conversational AI
- Basic workflow design
- AI agents
- APIs at a conceptual level
- Structured data
- Testing
- Customer support processes

No advanced telephony programming should be expected.

---

4. Tools and Technologies

Participants may use voice-agent platforms such as:

- Vapi
- Retell AI
- ElevenLabs conversational agents
- Bland AI
- Similar voice-agent platforms

An LLM may be used through:

- OpenAI
- Claude
- Gemini

Optional tools:

- Google Sheets
- Airtable
- Supabase
- Calendar integration
- Zapier
- Make

Participants should select tools based on the complexity of their project.

---

5. Step-by-Step Instructions

Step 1: Select a Business Use Case

Pick one business.

Example:

“ABC Dental Clinic.”

Define the most common reasons customers call.

For example:

1. Clinic timings
2. Address
3. Treatment information
4. Consultation fee
5. Doctor availability
6. Appointment booking

Avoid trying to automate every possible customer-support request.

Step 2: Design the Conversation

Before building the agent, create a sample conversation.

Example:

Agent:

“Hello. Thank you for calling ABC Dental Clinic. How may I help you?”

Customer:

“I want to book an appointment.”

Agent:

“Certainly. May I know your name?”

Customer:

“Rahul.”

Agent:

“Thank you, Rahul. What type of consultation would you like?”

Customer:

“General dental check-up.”

Agent:

“What day would you prefer?”

This helps participants understand the required workflow.

Step 3: Create a Knowledge Base

Prepare a simple business information document.

Example:

Clinic Name:

ABC Dental Clinic

Address:

XYZ Road, Bengaluru

Timings:

Monday–Saturday

9 AM–7 PM

Services:

- General dentistry
- Root canal treatment
- Dental implants
- Teeth whitening

Consultation Fee:

₹500

The AI should answer questions using this information.

Step 4: Write the System Prompt

Define the agent's role carefully.

For example:

"You are the customer-support voice assistant for ABC Dental Clinic.

Your job is to answer common customer questions and assist with appointment enquiries.

Speak politely and clearly.

Keep responses short because this is a phone conversation.

Never invent information.

If information is unavailable, tell the customer that a staff member will assist them.

For medical emergencies, instruct the caller to seek appropriate professional emergency assistance rather than attempting to diagnose the situation."

Step 5: Create the Voice Agent

Configure:

- Agent name
- AI model
- Voice
- Language
- Greeting message
- System prompt
- Knowledge base

Initially test the agent through the platform's browser interface.

Step 6: Add FAQ Handling

Create at least **10–15 common customer questions**.

Examples:

"What are your timings?"

"Where are you located?"

"What services do you provide?"

"What is the consultation charge?"

"Are you open on Sunday?"

"Do I need an appointment?"

"Do you provide teeth whitening?"

Test whether the agent gives consistent answers.

Step 7: Add Information Collection

The agent should collect customer details where relevant.

Example:

- Name
- Phone number
- Requested service
- Preferred appointment day
- Preferred time

The agent should repeat important details for confirmation.

Example:

“Just to confirm, you would like a dental consultation on Friday at approximately 4 PM. Is that correct?”

Step 8: Save Customer Requests

For a beginner project, appointment enquiries can simply be stored in:

- Google Sheets
- Airtable

Example columns:

Customer Name

Phone

Service

Preferred Date

Preferred Time

Status

The AI does not necessarily need to complete a real booking.

Saving an appointment request is sufficient for the MVP.

Step 9: Add Human Escalation

The agent should understand when it should stop trying to solve the problem.

Examples:

Customer:

“I have already paid but my account shows unpaid.”

Agent:

“I’ll need our support team to check your account details. I’ll record your request for a team member to follow up.”

Human escalation should occur for:

- Complaints
- Payment disputes
- Sensitive requests
- Information not available in the knowledge base
- Repeated misunderstanding
- Situations requiring professional judgment

Step 10: Add Safety Rules

The agent should never:

- Invent information
- Promise unavailable services
- Provide unsafe professional advice
- Reveal private customer information
- Pretend an action was completed when it was not

Participants should explicitly include these restrictions in the system prompt.

Step 11: Connect a Phone Number

If supported by the selected platform, connect a test phone number.

Participants should then make real calls to the agent.

If obtaining a phone number is impractical, a browser-based voice conversation is acceptable for the basic project.

Step 12: Test Different Scenarios

Test at least 15–20 conversations.

Examples:

Normal Request

“What are your timings?”

Booking Request

“I want an appointment tomorrow.”

Unknown Question

“Do you provide a treatment not included in your knowledge base?”

Angry Customer

“Nobody answered my call yesterday!”

Unclear Speech

“Appointment... Friday... maybe evening...”

Topic Change

Start with timings and then ask about services.

Incorrect Information

Ask a question where the AI could potentially hallucinate.

Observe whether the agent follows the knowledge base.

Step 13: Improve the Conversation

Review recordings or transcripts.

Look for:

- Responses that are too long
- Repetition
- Unnatural language
- Incorrect answers
- Interruptions
- Failure to understand users
- Missing confirmations

Modify the prompt and workflow accordingly.

---

6. Expected Outcomes

The final project should include:

- Working AI Voice Agent
- Defined business use case
- Custom greeting
- Knowledge base
- FAQ handling
- Natural voice interaction
- Information collection
- Basic workflow
- Human escalation
- At least one stored customer request
- Testing documentation

Optional:

A live phone number.

---

7. Tips and Best Practices

Keep responses short.

Phone conversations are different from chatbot conversations.

Avoid paragraphs of explanation.

The agent should usually respond in one or two sentences.

Clearly define what the agent knows.

Clearly define what it does not know.

Never allow the agent to invent business policies.

Create a strong escalation mechanism.

Always confirm important information.

For example:

“So your preferred appointment is Saturday at 11 AM. Is that correct?”

Start with FAQs before attempting complicated integrations.

Build in stages:

FAQ Agent

↓

Information Collection

↓

Lead Storage

↓

Appointment Request

↓

Optional Live Booking

This keeps the project manageable.

OPTIONAL ADVANCED FEATURES

Participants who want to go further may add:

- Multilingual support
- Hindi + English conversations
- Live appointment booking
- CRM integration
- WhatsApp follow-up
- SMS confirmation
- Call summaries
- Sentiment detection
- Lead qualification
- Automatic call transfer
- Personalized returning-customer interactions

These should not be compulsory.

COMMON PROJECT SUBMISSION REQUIREMENTS

Regardless of which project a participant chooses, the final submission should contain the following.

1. Problem Statement

Approximately 200–300 words explaining:

- The problem
- Target users
- Current challenges
- Proposed AI solution

2. Solution Architecture

A simple flow diagram.

Example for AI Tutor:

Student

↓

Web Interface

↓

AI Model

↓

Learning Response

↓

Quiz / Feedback

↓

Progress Recommendation

Example for Voice Agent:

Customer Call

↓

Voice Agent

↓

AI Model + Knowledge Base

↓

Response

↓

Customer Request

↓

Google Sheet / Human Team

3. Working Prototype

A usable version of the solution must be demonstrated.

4. Prompts Used

Participants should submit the important prompts used for:

- Building the application
- Configuring the AI
- Generating content
- Debugging

This is particularly important because **effective AI interaction is one of the learning outcomes of the course**.

5. Testing Report

Participants should document at least 10 test scenarios.

Recommended format:

Test	User Action	Expected Result	Actual Result	Status
1	User asks a normal question	AI responds correctly	Correct response	Pass
2	User enters invalid input	AI handles it	Error handled	Pass
3	User asks unknown question	AI acknowledges limitation	Correct escalation	Pass

6. Challenges Faced

Participants should explain:

- What did not work initially?
- What errors occurred?
- How did AI help debug them?
- What did they change?

This is an important part of the project.

The goal is not to show that everything worked immediately.

The goal is to demonstrate **problem-solving and iteration**.

RECOMMENDED PROJECT PHILOSOPHY

Participants should follow one simple principle throughout the project:

Build small → Make it work → Test it → Improve it.

The goal of these projects is not to create the next ChatGPT, Duolingo, Call of Duty or enterprise call centre.

The goal is to demonstrate that participants can take a real-world idea and use modern AI tools to create a **working digital product**.

A simple product that works reliably is much stronger than a complicated product with many unfinished features.

The ideal final project should make the evaluator think:

“This may be a prototype, but I can clearly see how this could become a real product.”

Project Submission Report template

Report Structure

1. Cover Page	Learner Name, Email Id, Mobile Number & Project Title
2. Project Overview	Brief introduction and objective
3. Problem Statement	200–300 words covering the problem, target users, challenges, and proposed AI solution
4. Target Users	Who will use the solution and what problem they face
5. Proposed Solution	How the project addresses the identified problem
6. Key Features	Main features/functionality of the project
7. Solution Architecture	Flow diagram showing how the solution works
8. Tools & Technologies	AI models, development tools, APIs, databases, hosting platforms, etc.
9. Working Prototype	Prototype link + screenshots/demo details
10. Prompts Used	Important prompts used for development, configuration, content generation, and debugging
11. Testing Report	Minimum 10 test scenarios with expected result, actual result, and status
12. Challenges & Solutions	Problems encountered, errors, debugging approach, and improvements
13. Final Outcome	What was successfully built and key learnings

14. Future Enhancements	Optional features/improvements that could be added later
15. Project Demo	Demo/public URL wherever applicable

All the best for your Capstone Project...!